EventPilot

Autonomous Event Planning Team: Demonstration Guide

Purpose: Deliver a clear, credible demonstration of a React, FastAPI, Azure OpenAI, LangGraph, MCP, Langfuse, and SQLite event-planning system.

> **Demo story:** A wedding reception needs a practical, budget-aware plan for 350 guests. EventPilot delegates planning work, uses MCP tools for operational evidence, synthesizes a plan, validates it, and traces every AI/tool step.

1. Pre-Demo Checklist

1. Start the MCP server in Terminal 1:
   `cd backend && source .venv/bin/activate && python -m app.mcp.server`
2. Start the API in Terminal 2:
   `cd backend && source .venv/bin/activate && uvicorn app.main:app --reload --port 8001`
3. Start React in Terminal 3:
   `cd frontend && npm run dev`
4. Open the frontend at `http://127.0.0.1:5173` or the Vite port displayed in Terminal 3.
5. Open Langfuse in a second browser tab and keep the trace list ready.

2. Demonstration Input

Field	Value
Event name	Meera and Karthik Wedding Reception
Date	14 February 2027
Location	Whitefield, Bengaluru
Expected attendees	350
Budget	₹1,500,000
Objective	Plan an elegant one-day wedding reception with guest welcome, stage ceremony, dinner service, photography, family coordination, entertainment, and smooth crowd management.

Why this example works: it exercises venue capacity, catering, crowd-flow, parking, family coordination, communications, schedule design, budget control, and operational risk.

3. Presenter Script: Initial Plan

1. Open **New event**, enter the example brief, and save it.
2. Say: *"The event brief becomes the shared state for the planning workflow. The system does not just return a static checklist; it coordinates specialist responsibilities."*
3. Show the generated Overview, then open Logistics, Budget, Schedule, Marketing, and Risks.
4. Point out the final plan contains a venue recommendation, budget allocation, 12-hour agenda, communication readiness, and risk actions.

Explain What Happens Behind the Screen

Workflow stage	What it does
Logistics Agent	Uses the MCP venue-search tool to identify capacity-appropriate local options.
Budget Agent	Uses MCP budget validation to confirm spending remains within the approved limit.
Schedule Agent	Uses MCP schedule conflict checking for 12-hour agenda quality.
Marketing Agent	Creates approval-gated communication and RSVP readiness steps.
Risk & Operations Agent	Builds an initial risk register for crowd flow, AV, RSVP variance, and ceremony timing.
Event Manager	Synthesizes all specialist context through Azure OpenAI into one structured plan.
Review Node	Checks budget limit, budget arithmetic, and time format; it permits one reflection retry if needed.

4. Show the Langfuse Trace

1. Refresh Langfuse and open the newest `eventpilot-plan-generation` trace.
2. Show the Azure model generation, prompt, structured Pydantic output, latency, token count, and cost.
3. Expand the tree and show MCP tool observations: `mcp.venue_search`, `mcp.budget_validation`, `mcp.schedule_conflict_check`, `mcp.marketing_readiness`, and `mcp.operational_risk_assessment`.
4. Explain that each tool span records input, output, timing, and errors without exposing hidden reasoning.

> **Key message:** Langfuse makes the multi-agent workflow inspectable. It shows what the manager used, which tools were called, the model response, and the cost/latency of a planning run.

5. Demonstrate Iterative Refinement

1. Open the **Budget** tab.
2. Click **Reduce budget by 20%**.
3. Explain that the available budget becomes **₹1,200,000**. The forecast can be lower because the system preserves a contingency reserve.
4. Show revised budget values and updated agent activity in the frontend.
5. Open Langfuse and select the newest `eventpilot-budget-revision` trace.
6. Confirm its prompt includes `Budget in INR: 1200000`.

Expected Outcome

- A forecast at or below ₹1,200,000.
- Reduced non-essential spend while retaining contingency.
- A distinct budget-revision trace correlated with the event session.
- Updated planning output persisted in SQLite.

6. Demonstrate MCP Independently

Use the MCP Inspector for a short technical proof:

1. Run `npx -y @modelcontextprotocol/inspector`.
2. Connect using Streamable HTTP at `http://127.0.0.1:8000/mcp`.
3. Call `venue_search` with Whitefield, 350 attendees, and ₹1,500,000.
4. Show returned venue capacity, rental, and features.

7. Demonstrate Human Approval

1. In the Plan workspace, open **Risks**.
2. Click **Set fallback** for the first risk and **Monitor** for the second risk.
3. Explain that the system records human decisions; it does not book venues, send communications, or take external actions automatically.
4. Return to Overview. The open-risk count should become zero and the status should change to **Ready to approve**.
5. Click **Approve plan**. The plan is marked approved and can be exported.
6. Click **Export plan** to demonstrate the final user-owned blueprint.

> **Approval message:** Agents can research, validate, recommend, and draft. A human remains responsible for risk decisions, final approval, vendor booking, and external communication.

8. Complete Flow Verification Checklist

Checkpoint	Expected evidence

Frontend	Event brief saves and generated plan displays in all tabs.
MCP	Inspector lists six tools and returns structured tool output.
Backend	`/health` returns `{"status":"ok","provider":"azure"}`.
Langfuse	Plan-generation, MCP tool observations, Azure generation, latency, tokens, and cost are visible.
Revision	Budget reduction produces `eventpilot-budget-revision` with the revised budget.
Approval	Risk actions are recorded; approval is enabled only after open risks are resolved.
Persistence	After FastAPI restart, the saved event and latest plan remain retrievable.

9. Closing Statement

“EventPilot demonstrates a manager-led, stateful event-planning workflow. Specialist agents use controlled MCP tools, the manager synthesizes a structured plan, a review loop guards basic constraints, SQLite preserves outputs, and Langfuse provides traceability for every significant AI and tool operation.”